UNITÀ 2 · WEB SERVICE · LEZIONE 8

I web service con REST

Niente buste XML e niente contratti di cento righe: risorse, indirizzi e i verbi di HTTP. Lo stile con cui oggi parlano quasi tutte le app.

5G TPSI Fielding, risorse, URI, metodi, codici di stato, JSON

PER COMINCIARE

Quale di queste API hai già usato, anche senza saperlo?

SONDAGGIO

- **A** Le previsioni del meteo in un'app
- **B** Il pulsante "Accedi con Google"
- **C** Una mappa dentro un sito
- **D** Un pagamento online
- **E** Un chatbot dentro un'app
- **F** Nessuna, che io sappia

UNA TESI DI DOTTORATO DEL 2000

REST nasce per descrivere il Web

> **Il nome Representational State Transfer vuole evocare l'immagine di come si comporta un'applicazione web ben progettata: una rete di pagine web [...] in cui l'utente avanza selezionando dei link.**

Roy T. Fielding, coautore delle specifiche di HTTP/1.1 e degli URI, nella tesi di dottorato all'Università della California a Irvine, 2000

REST non è un protocollo, né un formato, né una libreria: è uno **stile architetturale**, un insieme di vincoli. HTTP è il protocollo che lo realizza meglio.

DALLA LEZIONE 6

SOAP chiama operazioni, REST manipola risorse

	SOAP	REST
Al centro	Operazioni: `getLibro`, `prestaLibro`	Risorse con un URI: `/libri/3`
Azioni	Il nome dell'operazione è nel messaggio	I metodi di HTTP: GET, POST, PUT, DELETE
Messaggio	Envelope XML, sempre	Rappresentazioni: JSON, XML, HTML, immagini
Contratto	WSDL, obbligatorio	Facoltativo, di solito OpenAPI
HTTP	Solo un trasporto, quasi sempre POST	Semantica sfruttata: metodi, codici, header, cache
Dove vive	Banche, PA, sistemi aziendali	App, siti, cloud, API pubbliche

I VINCOLI DI FIELDING

Sei vincoli, uno sopra l'altro

Vincolo	Che cosa chiede
Client-server	Interfaccia separata dai dati: client e server evolvono in modo indipendente
Stateless	Ogni richiesta contiene tutto ciò che serve: il server non ricorda la "conversazione"
Cache	Ogni risposta dice se e per quanto si può riusare
Interfaccia uniforme	Risorse con URI, rappresentazioni, messaggi autodescrittivi, link (HATEOAS)
Sistema a livelli	Il client non sa se parla con il server o con un proxy, una CDN, un gateway
Codice su richiesta	Facoltativo: il server può inviare codice da eseguire, come JavaScript

PUZZLE

Abbina ogni vincolo al beneficio che porta

Stateless	---	Un bilanciatore può mandare ogni richiesta a un server diverso
Cache	---	L'app mobile e il sito usano lo stesso back-end
Sistema a livelli	---	Un client che conosce HTTP sa già come usare qualsiasi risorsa
Interfaccia uniforme	---	Si può mettere una CDN davanti al server senza cambiare il client
Client-server	---	Meno richieste al server e risposte più rapide

RISORSE E URI

Gli URI sono nomi, i metodi sono verbi

Così no

- `GET /getLibro?id=3`
- `POST /cancellaLibro`
- `GET /libri/3/elimina`
- `POST /api?azione=nuovoPrestito`

Così sì

- `GET /libri/3`
- `DELETE /libri/3`
- `GET /libri?autore=Calvino`
- `POST /libri/3/prestiti`

- Nomi al plurale per le collezioni, un identificatore per il singolo elemento.
- Gerarchia nel percorso, filtri e ordinamenti nella query string.

DALLA LEZIONE 5, SUI METODI DI BIBLIOAPI

Sicuro, idempotente: due parole da sapere

Metodo e URI	Che cosa fa	Sicuro	Idempotente
`GET /libri`	Legge l'elenco	sì	sì
`GET /libri/3`	Legge un libro	sì	sì
`POST /libri`	Crea un libro, il server sceglie l'id	no	no
`PUT /libri/3`	Sostituisce tutto il libro 3	no	sì
`PATCH /libri/3`	Modifica solo alcuni campi	no	non garantito
`DELETE /libri/3`	Elimina il libro 3	no	sì

Sicuro: non cambia lo stato del server. **Idempotente**: ripetere la stessa richiesta lascia il server come dopo la prima volta.

QUIZ A TRABOCCHETTO

Per un timeout il client ripete tre volte `PUT /libri/3` e tre volte `POST /libri`. Quanti libri in più ci sono?

QUIZ

A Nessuno: il server riconosce da solo le richieste doppie

B Sei: ogni richiesta riuscita scrive qualcosa

C Uno: i POST uguali vengono uniti

D Tre: i PUT sostituiscono sempre lo stesso libro 3, ogni POST ne crea uno nuovo

I CODICI DI STATO, DA PROGETTISTI

La prima cifra dice di chi è il problema

2xx

200 OK
201 Created
204 No Content

3xx

301 spostato
304 Not Modified, usa la cache

4xx · client

400 richiesta errata
401 non autenticato
403 vietato
404 non trovato
409 conflitto

5xx · server

500 errore interno
502 gateway
503 non disponibile

Una buona API risponde **201** con l'header `Location: /libri/4` quando crea, **204** quando cancella, e mette nel corpo degli errori un JSON che spiega il motivo.

METTITI ALLA PROVA

Quale di queste scelte viola il vincolo stateless?

QUIZ

A Ogni richiesta porta il token nell'header Authorization

B Il server ricorda la pagina di ogni utente: GET /libri/avanti dà la successiva

C L'elenco si sfoglia con GET /libri?pagina=3

D Le risposte contengono Cache-Control: max-age=60

AL COMPUTER · 15 minuti

Una vera API REST, nel browser

1. Apri nel browser: https://api.open-meteo.com/v1/forecast?latitude=41.90&longitude=12.50&current=temperature_2m
2. Leggi il JSON: trova la temperatura e la sua unità di misura.
3. Cambia latitudine e longitudine con quelle della tua città.
4. Apri gli Strumenti per sviluppatori (F12), scheda **Rete**, ricarica: annota codice di stato e `Content-Type`.
5. Togli la latitudine dall'URL e ricarica. Annota tutto in **meteo-api.txt**.

Controllo: con l'URL giusto lo stato è **200**, il tipo `application/json`, e il JSON ha un oggetto `current` con `temperature_2m` e un oggetto `current_units` con `°C`. Senza latitudine il server risponde **400** con un messaggio d'errore.

RAPPRESENTAZIONI

Una risorsa, più rappresentazioni

```
GET /api/libri/2 HTTP/1.1
Accept: application/json
Authorization: Bearer eyJhbGciOi…

HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: max-age=60

{ "id": 2,
  "titolo": "Il barone rampante",
  "autore": "Italo Calvino",
  "anno": 1957 }
```

- Il client chiede un formato con `Accept`, il server dice quale manda con `Content-Type`.
- **JSON**: oggetti, array, stringhe, numeri, `true`, `false`, `null`. Niente commenti.
- **Stateless** in pratica: il token viaggia in ogni richiesta, il server non ricorda chi sei.
- `Cache-Control` realizza il vincolo cache.

AL COMPUTER · 20 minuti

Leggere le risposte con curl

```
curl -i https://jsonplaceholder.typicode.com/users/1
curl -i https://jsonplaceholder.typicode.com/users/99
curl -i -X DELETE https://jsonplaceholder.typicode.com/posts/1
curl -i -X POST -H "Content-Type: application/json" -d "{\"title\":\"prova\"}" https://jsonplaceholder.typicode.com/posts
```

1. Apri il Prompt dei comandi (in PowerShell scrivi `curl.exe`).
2. Esegui i quattro comandi, uno alla volta.
3. In **curl-rest.txt** annota per ognuno metodo, URI, codice e un header della risposta.
4. Ripeti il POST: che id ottieni?

Controllo: utente 1: **200**, `Leanne Graham`. Utente 99: **404**, corpo `{}`. DELETE: **200**. POST: **201**, `"id": 101`, anche la seconda volta: il servizio finge di salvare.

QUANTO È REST LA TUA API?

Metti in ordine i livelli di maturità, dal più basso

1. Livello 1: un URI per ogni risorsa
2. Livello 3: le risposte contengono i link alle azioni possibili (HATEOAS)
3. Livello 0: un solo URL, tutto in POST, come in SOAP
4. Livello 2: metodi HTTP e codici di stato usati correttamente

Fielding, 2008: «Se il motore dello stato dell'applicazione non è guidato dall'ipertesto, allora non può essere RESTful e non può essere un'API REST. Punto.»

AL COMPUTER · 25 minuti

Progetta BiblioAPI

La biblioteca della scuola vuole un'API per i libri e per i prestiti. In un file **biblioapi-design.md** scrivi una tabella con: metodo, URI, corpo della richiesta, codici di risposta possibili.

1. Leggere tutti i libri, uno solo, quelli di un autore.
2. Aggiungere, sostituire ed eliminare un libro.
3. Prestare un libro e restituirlo.
4. Scrivi l'esempio JSON di un libro e di un errore.

Controllo: almeno 8 righe. `POST /api/libri` → 201 o 400. `DELETE /api/libri/{id}` → 204 o 404. Il prestito di un libro già prestato → **409**. Nessun verbo dentro gli URI.

PONTI VERSO OGGI

Le API dell'intelligenza artificiale sono REST

```
POST https://api.anthropic.com/v1/messages
x-api-key: LA_TUA_CHIAVE  (mai nel codice)
anthropic-version: 2023-06-01
content-type: application/json

{ "model": "nome-del-modello",
  "max_tokens": 300,
  "messages": [ { "role": "user",
    "content": "Spiega REST in due righe" } ] }
```





SFIDA A SQUADRE

Apri il lucchetto di BiblioAPI



- 1ª cifra: quanti sono i vincoli REST obbligatori
- 2ª cifra: la prima cifra del codice che dice "il libro non esiste"
- 3ª cifra: l'ultima cifra del codice di risposta a un POST che crea un libro
- 4ª cifra: l'ultima cifra del codice di risposta a una DELETE riuscita senza corpo

`codice`

BIGLIETTO D'USCITA

Completa prima di uscire

REST è uno stile architetturale descritto da Roy _____ nel 2000.

Il server non ricorda le richieste precedenti: REST è _____.

Una richiesta che ripetuta lascia il server nello stesso stato si dice _____.

Quando un POST crea una risorsa il server risponde con il codice _____.

L'URI giusto per eliminare il libro 7 è DELETE _____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- REST è uno stile con sei vincoli, uno facoltativo
- Risorse con URI, rappresentazioni in JSON
- Metodi HTTP come verbi; GET, PUT, DELETE idempotenti, POST no
- 201 + Location, 204, 400, 404, 409: codici che parlano
- Stateless: il token viaggia in ogni richiesta
- Le API dell'AI sono richieste HTTP con JSON

> **All'esame**: nella prova scritta si chiede spesso di progettare un'architettura client-server. Elencare gli endpoint con metodo, URI e codici, come in BiblioAPI, rende il progetto chiaro e verificabile.

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso.